QOTP – Quite OK Transport Protocol, version 0

QOTP is an encrypted, multiplexed transport over UDP. Every packet is authenticated; there is one cipher suite and no negotiation. A connection carries any number of independent byte streams, each reliable or best-effort.

All integers are little-endian. All sizes are bytes. The companion document, *QOTP – Why*, explains the reasoning behind each mechanism; references of the form "Why, A.n" point there.

1. Packet

Every packet is one UDP datagram:

```
plaintext header   type byte + 0..72   <- AAD
encrypted sn       6
encrypted payload  variable
MAC                16
```

The plaintext header always starts with the type byte:

```
bit  7 6 5 | 4 3 2 1 0
     type  |  version
```

`version` is 0. `type` selects one of five messages:

t	name	meaning
0	InitSnd	handshake, unencrypted
1	InitRcv	reply to InitSnd
2	InitCryptoSnd	handshake, 0-RTT
3	InitCryptoRcv	reply to InitCryptoSnd
4	Data	after the handshake

The whole plaintext header is the AEAD's additional data: readable by anyone, modifiable by no one.

2. Handshake

Two flows. Both derive a shared secret from an X25519 exchange of ephemeral keys, and both reach forward secrecy after one round trip.

In-band (1-RTT). The initiator knows only the address.

```
i -> r  InitSnd  [ep_pub_i, id_pub_i,
                  maxPayload]  pad to 1232
r -> i  InitRcv  [ep_pub_r, id_pub_r] + payload
i -> r  Data     ...
```

0-RTT. The initiator already knows the responder's identity public key, so it can encrypt application data in the first packet.

```
i -> r  InitCryptoSnd  [ep_pub_i, id_pub_i]
                       + payload, pad to 1232
r -> i  InitCryptoRcv  [ep_pub_r] + payload
i -> r  Data           ...
```

InitCryptoSnd is encrypted to `ECDH(ep_priv_i, id_pub_r)`, a long-lived key, so **that one message has no forward secrecy**. Every later message uses `ECDH(ep_priv, ep_pub)` and does.

Both initiating messages are padded to 1232 bytes so a small packet cannot elicit a large reply. A responder MUST reject a shorter one.

The **connection ID** is the first 8 bytes of the initiator's ephemeral public key. It is in the clear on every packet after InitSnd, so a receiver can select the connection, and therefore the key, before decrypting.

The handshake completes when the initiator receives a reply, or the responder receives its first Data packet.

3. Message layouts

Offsets are from the start of the datagram. `sn` is the encrypted sequence number (§4).

InitSnd – exactly 1232 bytes, no encryption.

```
0       type
1..32   ephemeral pubkey (initiator)
33..64  identity pubkey (initiator)
65..66  maxPayload (u16)
67..    zero padding
```

InitCryptoSnd – exactly 1232 bytes.

```
0       type
1..32   ephemeral pubkey (initiator)
33..64  identity pubkey (initiator)
65..70  sn
71..    [fillLen u16][filler][payload][MAC]
```

InitRcv – at least 105 bytes.

```
0       type
1..8    connection ID
9..40   ephemeral pubkey (responder)
41..72  identity pubkey (responder)
73..78  sn
79..    payload + MAC
```

InitCryptoRcv – at least 73 bytes.

```
0       type
1..8    connection ID
9..40   ephemeral pubkey (responder)
41..46  sn
47..    payload + MAC
```

Data – at least 42 bytes.

```
0       type
1..8    connection ID
9..14   sn
15..    payload + MAC
```

A receiver MUST reject a packet shorter than its minimum, which is `header + 6 (sn) + 16 (MAC) + 11 (smallest payload, §5)`.

4. Encryption

One suite: X25519, ChaCha20-Poly1305 and XChaCha20-Poly1305. No negotiation.

The payload is sealed with ChaCha20-Poly1305 under the shared secret, with the plaintext header as additional data, using a deterministic 12-byte nonce:

```
byte   0      1..5         6..11
     [dir]  [ zero ]  [ sn 48-bit ]
```

`dir` is bit 7 of byte 0: 1 when the sender opened the connection, 0 otherwise. The two directions therefore never share a nonce under one key.

The sequence number is then encrypted separately, so it is not a plaintext correlator. It is sealed with XChaCha20-Poly1305 under the same secret, taking its 24-byte nonce from the first 24 bytes of the already-sealed payload; only the first 6 bytes of the result go on the wire. A packet must therefore carry at least 24 bytes of sealed payload, which the minimum sizes in §3 guarantee.

To decrypt: recover `sn` (ChaCha20 keystream, counter 1), rebuild the nonce, open the payload. A receiver holding several secrets during a key change tries each; a wrong secret fails the MAC.

Key rotation. At sequence number `2^46` the sender generates a fresh ephemeral key and attaches it to a packet (`isKeyUpdate`, §5); the peer replies with its own (`isKeyUpdateAck`). At `2^47` both switch to the new secret and reset the sequence number to 0. The previous and next secrets stay accepted across the change, so packets in flight are not lost. Both sides may rotate at once, so one packet may carry both.

5. Payload

Inside the encryption, every packet carries a transport header. Sizes in bytes, in wire order:

```
flags        1  always
maxPayload   2  always
rcvWnd       1  always
ack          9  if hasAck    (12 if extend)
keyUpdate   32  if isKeyUpdate
keyUpdAck   32  if isKeyUpdateAck
streamHdr    7  if hasStream (10 if extend)
data        ..
```

```
flags bit
0  hasAck         ACK block present
1  hasStream      stream header present
2  extend         48-bit offsets, not 24
3  isClose        final byte (FIN)
4  isKeyUpdate    32-byte pubkey follows
5  isKeyUpdateAck 32-byte pubkey follows
6  reserved       MUST be 0
7  reserved       MUST be 0
```

`maxPayload` (u16) is the sender's largest acceptable UDP payload, on every packet so an MTU change needs no "already announced" state. The connection MTU is `max(min(local, remote), 1232)`: a bound the endpoints agree on, not a property of the path, which may drop larger packets silently. A sender MUST be able to fall back to 1232 (Why, A.3).

`rcvWnd` (u8, §7) is on every packet for the same reason, and because it describes the connection, not one stream.

ACK block – one packet acknowledged per block.

```
0..3  stream ID (u32)
4..6  offset (u24; u48 if extend)
7..8  length (u16)
```

Stream header.

```
0..3  stream ID (u32)  bit 31 = best-effort
4..6  offset (u24; u48 if extend)
7..   data
```

A packet with `hasStream` and no data is a probe: it is acknowledged like any other, and is how a side that only receives obtains an RTT sample. A packet without `hasStream` MUST have no trailing bytes.

The smallest payload is 11 bytes: flags, `maxPayload`, `rcvWnd`, and a stream header with a 24-bit offset.

6. Streams

A stream is an independent byte stream inside a connection, identified by an application-chosen `u32` in `0..2^31-1`. Bit 31 of the wire stream ID is reserved.

Reliability is a property of the stream, not of the packet. Bit 31 set means best-effort: the sender never retransmits that stream's data, and the receiver, once a head-of-line gap has been open longer than a gap timeout, skips it and continues. It travels in the ID so every packet of the stream carries it – a flag announced once could be lost, and a best-effort stream never retransmits the announcement.

Close and key updates are retransmitted even on a best-effort stream.

Each direction closes independently. `isClose` marks the last byte; a stream is gone once both directions have closed and no acknowledgements are outstanding.

7. Loss, flow and congestion

Acknowledgement. Each ACK names one packet by `(stream, offset, length)`. A packet is retransmitted when its timeout expires, or when three later packets have been acknowledged. A sender gives up after a bounded number of attempts (Why, A.6).

Receive window. Free space in the receiver's buffer, connection-wide, on every packet (§5). An 8-bit logarithm: 8 steps per power of two, 0 B to about 896 GB. True only when built, so a sender MUST take it from the newest packet seen, ordered by sequence number (§4).

```
enc 0  -> 0
enc 1  -> 128
else, a = enc - 2:
  base  = 1 << (a/8 + 8)
  value = base + (a mod 8) * base/8
```

A sender MUST NOT put more unacknowledged stream bytes in flight than the window. A receiver MUST accept in-order data even when full, or reassembly deadlocks. Retransmissions are exempt; while the window is closed a sender SHOULD retransmit only its lowest unacknowledged offset, the one packet the receiver is sure to take (Why, A.5).

A blocked sender MUST NOT go quiet: the window arrives only in a packet, and a peer with nothing to say sends none. It sends an empty packet once per RTO until the window reopens, never giving up. A receiver SHOULD send one unprompted when it drops a packet for lack of space or drains well below what it announced; that may be lost, so it does not replace the probe.

Congestion control. A sender paces packets: it measures the rate at which its bytes are acknowledged and spaces sends to match. Both the estimate and the pacing count **bytes on the wire**, headers included – mixing payload and wire bytes between the two makes them drift apart. A packet carrying no payload yields an RTT sample but never a bandwidth sample: it was sent because there was nothing else to send, so its rate would measure the sender's idleness, not the link.

A receive-only connection has no bandwidth estimate: it has nothing to pace.

The estimator is an implementation matter, not part of this specification. What is required: pace, do not burst; measure in wire bytes; and never let the scheduled send time run unboundedly ahead of the clock.

8. Constants

value	meaning
0	protocol version
1232	MTU floor; padded size of initiating packets
32	X25519 public key
8	connection ID
6	sequence number (48-bit)
16	Poly1305 MAC
11	smallest payload
42	smallest Data packet
2^31-1	largest stream ID
2^46	sn that begins a key change
2^47	sn that completes it

9. Security notes

- InitCryptoSnd is encrypted to a **public** identity key, so anyone who can dial chooses its plaintext. Parse it as hostile input; bound every length in it.
- The connection ID and packet type are readable on the wire. Everything else – stream IDs, offsets, the sequence number – is not.
- InitSnd carries `maxPayload` unencrypted and cannot authenticate it, so an on-path attacker can alter it. The authenticated `maxPayload` in every later packet corrects it.
- Nonce reuse is prevented by rotating keys before the 48-bit sequence number can wrap, and by the direction bit.